CCC181 – Application Development and Emerging Technologies(100 pts)

To be checked face to face score varies to how close you answer correctly 2 questions.

LABORATORY EXERCISE NO. 3

Laravel User Authentication and Registration

Course: Application Development and Emerging Technologies
Framework: Laravel
Development Environment: XAMPP, PHP, Composer, Node.js, npm, and MySQL
Existing Project: app-crud

I. Objectives

At the end of this laboratory exercise, students should be able to:

1. Install and configure Laravel Breeze in an existing Laravel application.
2. Implement user registration, login, logout, and password recovery features.
3. Customize the appearance of Laravel's authentication pages.
4. Configure email settings for password reset and other authentication-related notifications.
5. Protect Product CRUD routes using Laravel authentication middleware.
6. Test the application's authentication and access control functionality.

II. Required Software and Materials

- XAMPP with Apache and MySQL
- Visual Studio Code
- PHP and Composer
- Node.js and npm
- An existing Laravel app-crud project
- A web browser
- An email account for testing email functionality

III. Laboratory Procedures

Part A. Open the Existing Laravel Project

1. Open Visual Studio Code.
2. Open your existing app-crud project folder.
3. Open a terminal in the project directory.
4. Check the Laravel version:

php artisan --version

5. Start MySQL using the XAMPP Control Panel.
6. Verify that your .env file contains the correct database credentials.
7. Make a backup of your project before installing the authentication scaffolding.

Note: Do not create a new Laravel project. This laboratory continues the Product CRUD application developed in the previous exercise.

Part B. Install Laravel Breeze

Laravel Breeze provides a premade authentication system that includes login, registration, password reset, and other authentication features.

Step 1. Install Laravel Breeze

Run the following command in the terminal:

composer require laravel/breeze --dev

Wait for Composer to finish installing the package.

Step 2. Generate the authentication scaffolding

Run:

php artisan breeze:install

Follow the interactive prompts.

- Select **Blade** as the frontend stack.
- Select your preferred dark-mode option when prompted.
- Select the testing framework offered by your installed Breeze version, such as PHPUnit or Pest, if prompted.

Press Enter to confirm the appropriate choices.

Step 3. Install frontend dependencies

Run:

npm install

npm run build

These commands install the required frontend packages and compile the application's CSS and JavaScript assets.

Step 4. Run database migrations

Run:

php artisan migrate

This creates any pending database tables required by the authentication system, including the users table if its migration has not yet been executed.

Important: Existing migrations and database tables should be reviewed before making database changes. Do not delete your existing product database.

Part C. Run the Laravel Application

1. Open a terminal in Visual Studio Code.
2. Run the development server:

php artisan serve

3. Keep this terminal running.
4. If necessary, open another terminal and run:

npm run dev

Keep this second terminal running while using the Vite development server. Alternatively, use npm run build to compile the assets without running the development server.

5. Open the application in your browser:

http://127.0.0.1:8000

6. Observe the available **Login** and **Register** links.

Expected output: The application displays the authentication interface generated by Laravel Breeze.

Part D. Test User Registration and Login

Activity 1: Register a new user

1. Click the Register link.
2. Enter your name.
3. Enter a valid email address.
4. Create a password and confirm it.
5. Submit the registration form.
6. Observe the application's response after registration.

Activity 2: Log in

1. Log out if you are already logged in.
2. Navigate to /login.
3. Enter your registered email address and password.
4. Click Log in.
5. Verify that you can access the authenticated part of the application.

Activity 3: Test the available authentication pages

Inspect the following pages and identify their purposes:

Page	Purpose
/login	Allows registered users to log in
/register	Allows new users to create accounts
/forgot-password	Starts the password recovery process
/reset-password/{token}	Provides the password reset form through a valid reset link
/confirm-password	Requests password confirmation before sensitive actions
/verify-email	Supports email verification when enabled and configured

Some features, such as email verification, require additional configuration before they are fully operational.

Part E. Configure Email Settings

Laravel uses a mail service to send password reset links and authentication-related notifications.

Step 1. Open the .env file

Configure the mail settings using the credentials and connection details of your chosen email provider.

For example, a development setup using a mail testing service may look like this:

MAIL_MAILER=smtp

MAIL_HOST=your-smtp-host

MAIL_PORT=587

MAIL_USERNAME=your-smtp-username

MAIL_PASSWORD=your-smtp-password

MAIL_ENCRYPTION=tls

MAIL_FROM_ADDRESS="your-verified-sender@example.com"

MAIL_FROM_NAME="${APP_NAME}"

Replace the example values with the actual SMTP settings provided by your email service.

Security reminder: Never share your email password or SMTP credentials publicly or commit them to a public repository. For Gmail, use the appropriate supported SMTP configuration and an App Password when required; your regular account password may not work.

Step 2. Clear cached configuration

Run:

php artisan config:clear

Step 3. Test password recovery

1. Open /forgot-password.
2. Enter the email address of a registered user.
3. Submit the form.
4. Check the mailbox and spam folder for a password reset email.
5. Open the reset link and follow the instructions.

If no email arrives, verify the SMTP host, port, encryption, sender address, credentials, and mail service logs.

Expected output: When the mail service is correctly configured, the application can send a password reset notification to the registered email address.

Part F. Protect Product CRUD Routes Using Authentication Middleware

Laravel Breeze provides the authentication functionality. However, the routes for your existing Product CRUD application must also be protected.

Step 1. Open routes/web.php

Import the ProductController:

```
use App\Http\Controllers\ProductController;
use Illuminate\Support\Facades\Route;
```

Step 2. Add the authentication middleware group

Place the routes from the previous laboratory inside the auth middleware group:

```
Route::middleware('auth')->group(function () {

    Route::get('/product', [ProductController::class, 'index'])
        ->name('product.index');

    Route::get('/product/create', [ProductController::class, 'create'])
        ->name('product.create');

    Route::post('/product', [ProductController::class, 'store'])
        ->name('product.store');

    Route::get('/product/{product}/edit', [ProductController::class, 'edit'])
        ->name('product.edit');
```

```
    Route::put('/product/{product}/update', [ProductController::class, 'update'])
        ->name('product.update');

    Route::delete('/product/{product}/destroy', [ProductController::class, 'delete'])
        ->name('product.destroy');

});
```

Keep the existing Breeze authentication routes, usually loaded through:

require __DIR__.'/auth.php';

Do not remove existing routes that your application still needs.

Important: The delete route above uses the delete() method from the ProductController developed in the previous laboratory. If your controller uses a method named destroy(), update the route to use that method instead.

Step 3. Verify the routes

Run:

php artisan route:list

Check that the product routes are registered and protected by the auth middleware.

Part G. Test Authentication Middleware

Perform the following tests.

Test 1: Access without login

1. Log out of the application.
2. Navigate to http://127.0.0.1:8000/product.
3. Observe what happens.

Expected result: Laravel redirects the unauthenticated user to the login page.

Test 2: Access after login

1. Log in using your registered account.
2. Navigate to /product.

3. Verify that the product list is displayed.
4. Open the create, edit, update, and delete pages or actions as appropriate.

Expected result: An authenticated user can access the protected product routes, subject to the application's other validation and authorization rules.

Test 3: Logout

1. Log out of the application.
2. Attempt to revisit /product.
3. Verify that login is required again.

Expected result: The application prevents unauthenticated access to the protected routes.

Part H. Customize the Authentication Interface

Laravel Breeze provides Blade templates that can be customized to match the application's design.

1. Open the resources/views directory.
2. Locate the authentication-related Blade views and shared layouts generated by Breeze.
3. Customize the colors, logos, text, spacing, and page layout.
4. Keep the authentication form fields and their expected names intact.
5. Test the login and registration forms after making changes.

Expected output: The login and registration pages have a customized appearance while retaining their authentication functionality.

IV. Laboratory Deliverables

Submit the following:

1. Screenshot of the Laravel Breeze login page.
2. Screenshot of the registration page.
3. Screenshot showing a successfully registered user.
4. Screenshot of the Product CRUD page after login.
5. Screenshot showing the login redirect when a logged-out user accesses /product.
6. Screenshot showing the password reset email or a successful email test.

7. Copy of the modified routes/web.php.
8. A short explanation of how authentication middleware protects the Product CRUD application.

V. Guide Questions

Answer the following questions:

1. What is Laravel Breeze, and what problem does it solve?
2. What is the difference between registration and authentication?
3. What is the purpose of the auth middleware?
4. Why should Product CRUD routes be protected from unauthenticated users?
5. What is the role of SMTP configuration in password recovery?
6. Why should email credentials be kept in the .env file?
7. What is the difference between authentication and authorization?
8. Does placing a route inside the auth middleware group automatically make a user an administrator? Explain your answer.

VI. Assessment Rubric

Criteria	**Points**
Laravel Breeze installation and configuration User registration and login functionality	20
Product route protection using authentication middleware	20
Email configuration and password recovery test	15
Authentication page customization	10
Answered Questions	35
Total	**100**

VII. Conclusion

Laravel Breeze simplifies the implementation of user authentication by providing reusable login, registration, and password recovery features. By integrating Breeze into the existing app-crud application and protecting Product CRUD routes using authentication middleware, developers can restrict product management access to logged-in users while maintaining the application's existing functionality.